MODULE 08 | SYSTEM ADMINISTRATION

System Administration

Run a Linux server: users, services, logs, scheduled jobs and a firewall.

Mission	Topic	Commands / Concepts
1	Users, sudo and Packages	useradd, passwd, usermod, sudo, apt
2	Manage Services: systemctl	systemctl start/stop/restart/enable/status
3	Read the Logs: journalctl and /var/log	journalctl, -u, -f, --since, -p, /var/log
4	Schedule Jobs: crontab	crontab -e/-l/-r, cron syntax, at, timers
5	Firewall: ufw	ufw allow/deny/limit, status, logging, delete
Boss	Harden the Server	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

The sysadmin toolkit

Task	Tool
Manage users and privileges	`useradd`, `passwd`, `usermod`, `sudo`
Install software	`apt` (Debian/Ubuntu), `dnf` (Fedora/RHEL)
Start, stop, enable services	`systemctl`
Read logs	`journalctl`, `/var/log/`
Schedule jobs	`crontab`, `systemd timers`
Block and allow traffic	`ufw`, `iptables`, `nftables`
Check resources	`df`, `free`, `top`, `uptime`

Golden rules

- Work as a normal user and use `sudo` only when needed.
- Change one thing at a time and check the result.
- Keep a backup or a second login open before changing SSH or firewall rules.
- Read the logs first. They usually name the problem.
- Commands here need a real Linux machine or VM with systemd. Inside a container some may not work.

Warning
Examples on this page change real system settings. Practise on a virtual machine or a disposable server, not a production system.

MISSION 1 | +50 XP

Users, sudo and Packages

Goal: Create accounts, grant admin rights and install software.

```
$ sudo adduser alice                  # friendly, interactive (Debian/Ubuntu)
$ sudo useradd -m -s /bin/bash bob      # low-level: -m makes home, -s sets shell
$ sudo passwd bob                       # set or change a password
$ sudo usermod -aG sudo bob             # give admin rights (append to group)
$ sudo usermod -L bob                   # lock account
$ sudo usermod -U bob                   # unlock
$ sudo userdel -r bob                   # delete user and home folder
$ getent passwd alice                   # look up an account
$ last -5                               # recent logins
```

File	Contains
`/etc/passwd`	User accounts (name, UID, home, shell)
`/etc/shadow`	Password hashes (root only)
`/etc/group`	Group definitions and members
`/etc/sudoers`	Who may use sudo (edit only with `visudo`)

Warning
Never edit `/etc/sudoers` with a plain editor. A typo can lock you out of sudo. Use `sudo visudo`.

Packages (Debian/Ubuntu)

```
$ sudo apt update                     # refresh package lists
$ sudo apt upgrade                    # install available updates
$ sudo apt install nginx htop         # install
$ apt search "web server"             # search
$ apt show nginx                      # details
$ sudo apt remove nginx               # remove (keep config)
$ sudo apt purge nginx                # remove with config
$ sudo apt autoremove                 # remove unused dependencies
$ dpkg -l | grep nginx                # installed?
```

Tip
Fedora, RHEL and Rocky use `dnf install`, `dnf remove` and `dnf update` for the same jobs.

Practice

- Create a user `trainee` with a home directory and a password.
- Add `trainee` to a new group `devs` and verify with `id trainee`.
- Install `htop`, run it, then remove it.

MISSION 2 | +50 XP

Manage Services: systemctl

Goal: Control background services and make them start at boot.

```
$ systemctl status nginx
* nginx.service - A high performance web server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
     Active: active (running) since Sat 2026-11-14 09:12:00 IST; 2h ago
   Main PID: 842 (nginx)

$ sudo systemctl start nginx           # start now
$ sudo systemctl stop nginx            # stop now
$ sudo systemctl restart nginx         # stop and start
$ sudo systemctl reload nginx          # re-read config without dropping connections
$ sudo systemctl enable nginx          # start at every boot
$ sudo systemctl disable nginx         # do not start at boot
$ sudo systemctl enable --now nginx    # enable and start together
$ systemctl is-active nginx            # active / inactive
$ systemctl is-enabled nginx
```

Concept	Meaning
`start` vs `enable`	start affects **now**; enable affects **next boot**
`restart` vs `reload`	restart interrupts; reload applies config gracefully
`mask`	Completely prevent a service from starting

Listing and finding problems

```
$ systemctl list-units --type=service --state=running
$ systemctl --failed                   # which services failed?
$ systemctl list-unit-files --state=enabled
$ systemctl cat nginx                  # show the unit file
$ sudo systemctl daemon-reload         # after editing unit files
```

Your own service

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Python App
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
User=appuser

[Install]
WantedBy=multi-user.target

$ sudo systemctl daemon-reload
$ sudo systemctl enable --now myapp
```

Power control

```
$ sudo systemctl reboot
$ sudo systemctl poweroff
```

Practice

- Check the status of `ssh` (or `sshd`) and see whether it is enabled.
- List all failed services.
- Install `nginx`, start it, confirm port 80 with `ss -tuln`, then stop and disable it.

MISSION 3 | +50 XP

Read the Logs: journalctl and /var/log

Goal: Find out what happened by reading system and service logs.

```
$ journalctl                          # everything, oldest first
$ journalctl -r                       # newest first
$ journalctl -n 50                    # last 50 lines
$ journalctl -f                       # follow live (like tail -f)
$ journalctl -u nginx                 # one service
$ journalctl -u nginx -f              # follow one service
$ journalctl -b                       # since this boot
$ journalctl -b -1                    # previous boot
$ journalctl -p err                   # priority error and worse
$ journalctl --since "1 hour ago"
$ journalctl --since "2026-11-14 09:00" --until "2026-11-14 10:00"
$ journalctl -k                       # kernel messages
$ journalctl --disk-usage
```

Priority	Name
0	emerg
1	alert
2	crit
3	err
4	warning
5	notice
6	info
7	debug

Classic log files

File	Contents
`/var/log/syslog`	General system messages (Debian/Ubuntu)
`/var/log/auth.log`	Logins, sudo, SSH attempts
`/var/log/kern.log`	Kernel messages
`/var/log/dpkg.log`	Package installs
`/var/log/nginx/`	Web server access and error logs

```
$ sudo tail -f /var/log/auth.log
$ sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' \
    | sort | uniq -c | sort -rn | head
$ journalctl -u ssh --since today | grep -i failed
```

Tip
Combine this module with Module 03. `grep`, `awk`, `sort` and `uniq -c` turn raw logs into answers such as 'which IPs failed to log in most?'.

Keeping logs under control

```
$ sudo journalctl --vacuum-time=7d     # keep one week
$ sudo journalctl --vacuum-size=500M   # cap size
$ cat /etc/logrotate.conf              # rotation rules for /var/log files
```

Practice

- Show the last 20 log lines of the `ssh` service.
- Show only errors from the current boot.
- Find failed login attempts in `/var/log/auth.log` and count them per source IP.

MISSION 4 | +50 XP

Schedule Jobs: crontab

Goal: Run commands automatically at fixed times.

```
$ crontab -e              # edit your schedule
$ crontab -l              # list your jobs
$ crontab -r              # remove ALL your jobs (careful!)
$ sudo crontab -u bob -l  # another user's jobs
```

The five time fields

```
* * * * *  command
| | | | |
| | | | +--- day of week (0-7, Sunday = 0 or 7)
| | | +----- month (1-12)
| | +------- day of month (1-31)
| +--------- hour (0-23)
+----------- minute (0-59)
```

Schedule	Meaning
`* * * * *`	Every minute
`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Top of every hour
`30 2 * * *`	Every day at 02:30
`0 9 * * 1-5`	Weekdays at 09:00
`0 0 1 * *`	Midnight on the 1st of each month
`@reboot`	Once at startup
`@daily`	Once a day

A realistic crontab

```
MAILTO=""
PATH=/usr/local/bin:/usr/bin:/bin

# backup every night at 02:30
30 2 * * * /home/haani/backup.sh >> /home/haani/backup.log 2>&1
# clean temp files every Sunday at 03:00
0 3 * * 0 find /tmp -type f -mtime +7 -delete
# heartbeat every 5 minutes
*/5 * * * * curl -fsS https://example.com/ping > /dev/null
```

Why jobs fail
Cron runs with a tiny environment. Use **absolute paths**, set PATH in the file, make scripts executable, and always redirect output to a log. Check results in `journalctl -u cron` or `/var/log/syslog`.

Other schedulers

```
$ echo "/home/haani/once.sh" | at 23:00     # one-off job
$ atq                                        # list queued at-jobs
$ systemctl list-timers                      # systemd timers
```

Practice

- Add a job that appends the date to `~/cron_test.log` every minute, wait, then confirm and remove it.
- Write the cron line for 'every Monday at 07:15'.
- Schedule a nightly backup script with its output logged.

MISSION 5 | +50 XP

Firewall: ufw

Goal: Allow only the traffic you intend and block everything else.

`ufw` (Uncomplicated Firewall) is a friendly front end for the kernel's netfilter rules.

> **Warning**
> Allow SSH **before** enabling the firewall on a remote machine. If you do not, you will lock yourself out.

```
$ sudo ufw status verbose
Status: inactive
$ sudo ufw default deny incoming        # block unrequested inbound traffic
$ sudo ufw default allow outgoing      # allow all outbound
$ sudo ufw allow ssh                   # same as 22/tcp
$ sudo ufw allow 80/tcp                # web
$ sudo ufw allow 443/tcp               # https
$ sudo ufw enable                      # turn it on
$ sudo ufw status numbered
```

More rules

```
$ sudo ufw allow from 192.168.1.0/24 to any port 22   # SSH only from the LAN
$ sudo ufw deny 3306                                  # block MySQL
$ sudo ufw deny from 203.0.113.50                     # block one address
$ sudo ufw limit ssh                                  # rate-limit brute force
$ sudo ufw delete 3                                   # remove rule number 3
$ sudo ufw delete allow 80/tcp                        # remove by rule
$ sudo ufw logging on                                 # log to /var/log/ufw.log
$ sudo ufw disable                                    # turn off
$ sudo ufw reset                                      # wipe all rules
```

Policy	Effect
`allow`	Accept the traffic
`deny`	Drop silently (sender waits and times out)
`reject`	Refuse and tell the sender
`limit`	Allow, but block IPs making too many attempts

Verify from outside

```
$ sudo ss -tulnp                    # what is listening?
$ nc -zv server 22                  # test from another machine
$ sudo tail -f /var/log/ufw.log
```

> **Tip**
> Defence in depth: a firewall hides services you do not need exposed, but it does not replace updates, strong SSH keys and least-privilege users.

Practice

- Set default policies, allow SSH and HTTP, and view the numbered rules.
- Remove the HTTP rule by number.
- Rate-limit SSH, then check the status.

BOSS BATTLE | +150 XP

Boss: Harden the Server

Goal: combine everything from this module without hints.

Scenario

A fresh Ubuntu server needs to be made production-ready: a safe user, a running web service, a firewall, a nightly cleanup and log checks.

Tasks

1. Update the package lists and installed packages.
2. Create user `deploy` with a home folder and sudo rights.
3. Install `nginx`, enable it at boot and start it now.
4. Confirm nginx is active and listening on port 80.
5. Allow SSH, HTTP and HTTPS in `ufw`, set default deny incoming, then enable it.
6. Schedule a daily 03:00 cron job that deletes files in `/tmp` older than 7 days.
7. Show the last 10 log lines of nginx from the journal and any errors from this boot.
8. List failed services and show disk and memory usage.

Sample solution

```
sudo apt update && sudo apt upgrade -y
sudo adduser deploy && sudo usermod -aG sudo deploy
sudo apt install -y nginx
sudo systemctl enable --now nginx
systemctl is-active nginx && ss -tuln | grep ':80'
sudo ufw default deny incoming && sudo ufw default allow outgoing
sudo ufw allow ssh && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
sudo ufw enable && sudo ufw status verbose
(crontab -l 2>/dev/null; \
 echo '0 3 * * * find /tmp -type f -mtime +7 -delete') | crontab -
journalctl -u nginx -n 10; journalctl -b -p err
systemctl --failed; df -h; free -h
```

Check Your Understanding

Q1. Which command creates a user with a home directory?

Q2. How do you safely edit sudo rules?

Q3. Difference between `systemctl start` and `enable`?

Q4. How do you see logs of one service?

Q5. What does `journalctl -f` do?

Q6. What does `*/10 * * * *` mean?

Q7. Why use absolute paths in cron?

Q8. What must you allow before enabling ufw on a remote server?

Q9. Which ufw policy rate-limits brute force attempts?

Q10. What does `systemctl --failed` show?

Answers

#	Answer
1	useradd -m (or adduser).
2	sudo visudo
3	start runs it now; enable starts it at boot.
4	journalctl -u servicename
5	Follows new log lines live.
6	Every 10 minutes.
7	Cron has a minimal environment and PATH.
8	SSH (port 22).
9	ufw limit
10	Services that failed to start or crashed.

Quick Reference Cheat Sheet

Command	Purpose	Example
adduser / usermod -aG	Users and groups	sudo usermod -aG sudo bob
apt install / update	Packages	sudo apt install nginx
systemctl status	Service state	systemctl status ssh
systemctl enable --now	Boot + start	sudo systemctl enable --now x
systemctl --failed	Failed services	systemctl --failed
journalctl -u -f	Service logs	journalctl -u nginx -f

Command	Purpose	Example
journalctl -p err -b	Errors this boot	journalctl -p err -b
crontab -e / -l	Schedule jobs	30 2 * * * /bk.sh
ufw allow / deny	Firewall rules	sudo ufw allow 443/tcp
ufw status numbered	Review rules	sudo ufw status numbered
df -h / free -h	Disk and memory	df -h

Course complete!
You have finished all 8 modules. Revisit the boss battles, build your own scripts and try a real VM to cement these skills.